

UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO
FACULTAD DE CIENCIAS

ALMACENES Y MINERÍA DE DATOS

Triage de reportes de incidentes viales del C5

¿Es real o falso un reporte vial? Predicción de veracidad y
descubrimiento de corredores de incidentes en la Ciudad de México

Proyecto Final

Integrante: José Rubén Alfaro González
No. de cuenta: 320516436

Profesora: Jessica Santizo Galicia
Ayudante de Teoría: Diego Antonio Villalba González
Ayudante de Laboratorio: Ares Gael Castro Romero

Ciudad de México, junio de 2026

Índice

1. Introducción	2
2. Descripción del dataset	2
3. Limpieza y preparación de datos	3
4. Análisis Exploratorio de Datos (EDA)	4
4.1. Balance del target	4
4.2. Estadística descriptiva y valores atípicos	5
4.3. Distribuciones temporales	5
4.4. Geografía del reporte: ¿quién reporta y qué tan confiable es?	5
4.5. Síntesis del EDA	7
5. Modelo supervisado	7
5.1. Variable objetivo y variables de entrada	7
5.2. Proceso	7
5.3. Resultados	8
5.4. Importancia de variables	9
5.5. Análisis de errores	10
5.6. Experimento de fuga de información	10
5.7. Comparación con una red neuronal (MLP)	10
5.8. Modelo final ligero y persistencia	11
6. Agrupamiento (Clustering)	11
6.1. Propósito y elección del algoritmo	11
6.2. Preprocesamiento y selección de parámetros	11
6.3. Resultados	12
6.4. Evaluación e interpretación	13
7. Arquitectura y diseño	14
8. Reutilización del modelo	15
9. Conclusiones	15
10. Uso de Inteligencia Artificial Generativa	16

1. Introducción

Problema. El Centro de Comando, Control, Cómputo, Comunicaciones y Contacto Ciudadano (C5) de la Ciudad de México recibe cientos de miles de reportes de incidentes viales al año a través de distintos canales (llamadas al 911, botones de auxilio, cámaras, radio). Una fracción importante de esos reportes no corresponde a un incidente confirmado, sino suelen ser falsos o meramente informativos. Despachar unidades de emergencia a reportes que no lo ameritan desperdicia recursos escasos y retrasa la atención de incidentes reales.

Objetivo. Construir un modelo que, con la información disponible *al momento de recibir un reporte* (canal de entrada, tipo de incidente, hora, alcaldía y ubicación), estime la probabilidad de que sea un incidente real (confirmado, “afirmativo”) frente a falso/informativo, para apoyar la priorización del despacho, y de forma complementaria, identificar mediante agrupamiento espacial las *vialidades y/o cruces* donde se concentran los incidentes y de qué tipo, como insumo para la planeación operativa.

Justificación y valor. En nuestro conjunto de datos, el 36.5 % de los reportes viales no duplicados que recibe el C5 termina cerrándose como falso o informativo: más de uno de cada tres despachos potenciales no corresponde a un incidente verificable. En una ciudad donde las unidades de emergencia son un recurso finito, cada despacho innecesario tiene un costo de oportunidad directo sobre los incidentes verdaderos, cuya atención oportuna puede ser cuestión de vida o muerte (la literatura documenta el papel central de las llamadas de emergencia relacionadas con tránsito en la CDMX [2]). Un modelo de triage que ordene los reportes entrantes por su probabilidad de ser reales permite priorizar el despacho con evidencia, concentrar la verificación humana en los casos ambiguos y dimensionar el problema de las falsas alarmas. El análisis es, además, completamente reproducible sobre datos públicos oficiales.

2. Descripción del dataset

Fuente. Conjunto de datos abiertos “Incidentes viales reportados por el C5” del Portal de Datos Abiertos de la Ciudad de México (<https://datos.cdmx.gob.mx>), publicado por el propio C5 bajo licencia CC-BY-4.0. Corte 2022–2024, descargado en junio de 2026. La fuente es de uso académico reconocido: estudios recientes sobre siniestralidad vial en la CDMX la emplean explícitamente como una de sus fuentes primarias [3].

Tamaño. El archivo crudo contiene 504 261 reportes, que tras la preparación (Sección 3) el conjunto de trabajo queda en **289 885 reportes y 18 variables** con mezcla de variables numéricas y categóricas.

Variables. El diccionario completo (significado, tipo, valores posibles y rol de cada variable) está en `reports/diccionario_datos.md` y en el notebook `01_edu.ipynb`. La Tabla 1 resume las principales.

Cuadro 1: Variables principales (diccionario resumido).

Variable	Tipo	Descripción / valores
codigo_cierre	cat.	A=Afirmativo, D=Duplicado, F=Falso, I=Informativo (base del target)
REAL	0/1	Target: 1 si afirmativo, 0 si falso/informativo
tipo_entrada	cat. (9)	Canal: llamada del 911, botón de auxilio, cámara, radio, ...
incidente_c4	cat. (16)	Choque s/lesionados, choque c/lesionados, atropellado, ...
tipo_incidente_c4	cat. (7)	Accidente, lesionado, cadáver, ...
alcaldia_catalogo	cat. (16)	Las 16 alcaldías de la CDMX
HORA, FRANJA, dia_semana, MES, ANIO	num./cat.	Variables temporales del reporte
latitud, longitud	num.	Ubicación del incidente
TIEMPO_CIERRE_MIN	num.	Minutos hasta el cierre (<i>fuga</i> : excluida del modelo)
clas_con_f_alarma	cat.	Clasificación posterior al cierre (<i>fuga</i> : excluida del modelo)

Algunas limitaciones a tener en cuenta.

- El etiquetado de veracidad es *institucional*: refleja los criterios de cierre del C5, no una verificación independiente; cualquier sesgo de ese proceso lo hereda el modelo.
- La llamada del 911 concentra el 87.6 % del volumen, de modo que el desempeño global está dominado por el canal más ambiguo.
- La geocodificación es incompleta en una fracción mínima (366 reportes sin alcaldía, 0.13 %)
- La ventana temporal es 2022–febrero de 2024, por lo que los patrones podrían no extrapolar a periodos con dinámicas distintas.
- No se dispone de variables de contexto (clima, tráfico, contenido de la llamada) que probablemente elevarían el techo de predicción.

3. Limpieza y preparación de datos

La preparación se implementó con el **patrón de diseño Pipeline**: una secuencia de pasos atómicos (cada uno una función $df \rightarrow df$) encadenados por una clase `PipelinePreparacion` que registra una bitácora auditable de filas antes/después de cada paso (notebook `01_edu.ipynb`, sección 2).

Cuadro 2: Bitácora del pipeline de preparación.

Paso	Filas antes	Filas después
quitar_duplicados	504 261	289 935
quitar_anio_ruido	289 935	289 885
construir_target	289 885	289 885

Decisión clave — eliminación de duplicados. La distribución del cierre en el crudo es: D (duplicado) 42.5 %, A (afirmativo) 36.5 %, F (falso) 20.3 %, I (informativo) 0.7 %. Un reporte con cierre D corresponde a un *evento real* que ya fue reportado antes. Como el objetivo del modelo es distinguir reportes reales de falsos, y la deduplicación (verificar si ya hay una unidad en el sitio) es responsabilidad del *sistema de despacho* y no del modelo de veracidad, los duplicados se eliminan. El target se define sobre los reportes no duplicados: `REAL`=1 si el cierre es afirmativo (**A**), 0 si es falso

(F) o informativo (I). Se descartó la alternativa de contar los duplicados como reales porque produce un balance mucho peor ($\sim 79/21$) y un target conceptualmente borroso (mezcla “confirmado” con “ya reportado”), incrementando el riesgo de la tendencia al modelo Dummy.

Otras decisiones.

- El segundo paso elimina 50 reportes de diciembre de 2021 que se colaron en el corte (ruido de frontera).
- *Faltantes*: solo tres columnas tienen nulos y todos son marginales (`alcaldia_catalogo` 366 (0.13 %), `tipo_entrada` 3, `TIEMPO_CIERRE_MIN` 2). No se eliminan filas: el `ColumnTransformer` del modelo los imputa con la categoría más frecuente.
- *Duplicados exactos*: tras retirar los cierres D no queda ningún folio repetido ni fila idéntica
- *Outliers*: la regla IQR sobre `TIEMPO_CIERRE_MIN` marca 49 211 reportes (17.0 %) por encima del límite superior (287.5 min); no se eliminan porque son tiempos de servicio *reales* (casos complejos que tardan en cerrarse), no errores de captura, y porque esa columna no es variable del modelo.
- *Anti-fuga*: `TIEMPO_CIERRE_MIN` y `clas_con_f_alarma` se conservan en el dataset solo para análisis, pero quedan excluidas del modelo (Sección 5).

4. Análisis Exploratorio de Datos (EDA)

4.1. Balance del target

Tras la preparación, el 63.5 % de los reportes son reales (cierre afirmativo) y el 36.5 % falsos/informativos, esto muestra un desbalance moderado de 1.74:1 (Figura 1). De aquí se derivan dos decisiones de modelado: la métrica rectora es el **F1 macro** (ya que la *accuracy* la alcanzaría casi un clasificador trivial que diga “real” a todo) y el bosque usa `class_weight='balanced'` para no sacrificar el recall de la clase minoritaria, pues detectar reportes falsos es justamente el valor de negocio.

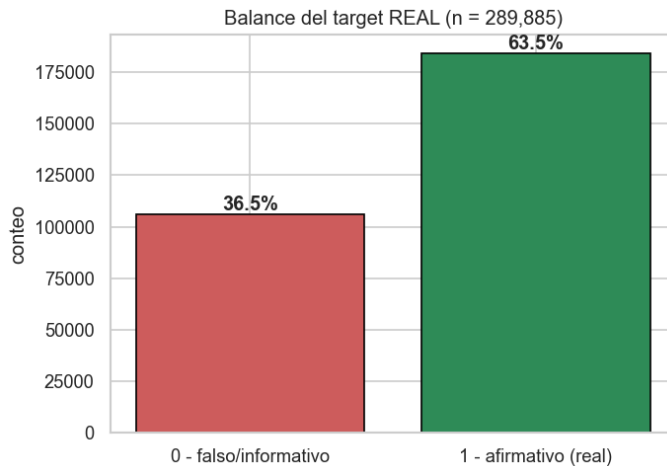


Figura 1: Balance del target REAL en el conjunto depurado (289 885 reportes).

4.2. Estadística descriptiva y valores atípicos

Para las dos numéricas más informativas se reportó media, mediana, desviación, cuartiles, rango y forma. **HORA** es casi simétrica (asimetría -0.52 , curtosis -0.58 ; mediana 15 h): los reportes se reparten a lo largo del día con concentración vespertina. En cambio **TIEMPO_CIERRE_MIN** está fuertemente sesgada a la derecha (asimetría 3.7, curtosis 15.5): mediana de ~ 189 minutos con una cola que llega a 4657. La regla IQR marca 17.0 % de atípicos en esa cola (límite superior 287.5 min) que se conservan por ser tiempos de servicio reales y porque la variable queda fuera del modelo por fuga. Esta forma extrema, típica de tiempos de servicio, ilustra por qué los atípicos deben evaluarse en su contexto y no eliminarse mecánicamente.

4.3. Distribuciones temporales

La Figura 2 muestra los ritmos de la ciudad: pocos reportes de madrugada (3–6 h), ascenso matutino y el grueso entre la tarde y la noche; por día de la semana destacan viernes y sábado (mayor movilidad y vida nocturna) y el lunes es el día más tranquilo. Son patrones plausibles que aportan señal temporal moderada al modelo.

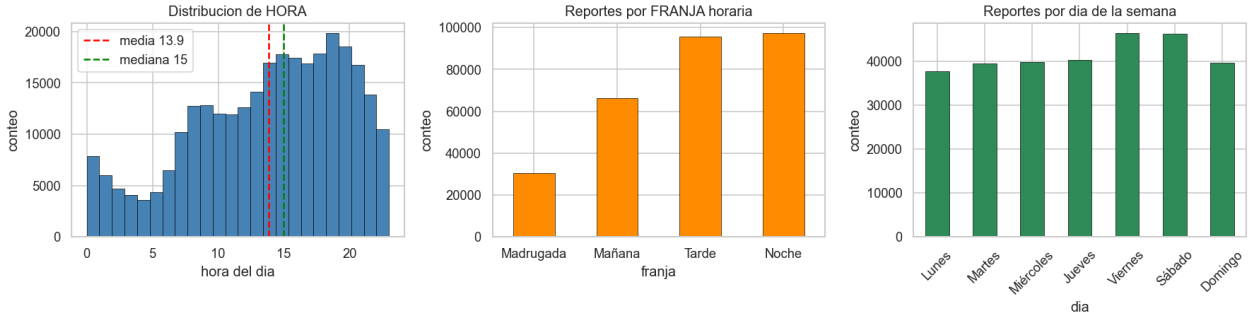


Figura 2: Distribución de reportes por hora del día, franja horaria y día de la semana.

4.4. Geografía del reporte: ¿quién reporta y qué tan confiable es?

Por canal de entrada. Los canales *institucionales* operados por personal del C5 confirman casi siempre con radio 93.3 %, cámara 92.9 % y botón de auxilio 88.4 %. Por otro lado, las llamadas del 911 (que concentra el 87.6 % del volumen) confirman solamente 59.8 %, y los aplicativos ciudadanos apenas 28.4 %. Esto muestra que lo que entra por canales institucionales puede despacharse casi automáticamente con alta prioridad, mientras que las verificaciones más precisas deben concentrarse en la avalancha de llamadas ciudadanas, donde hay mayor ambigüedad.

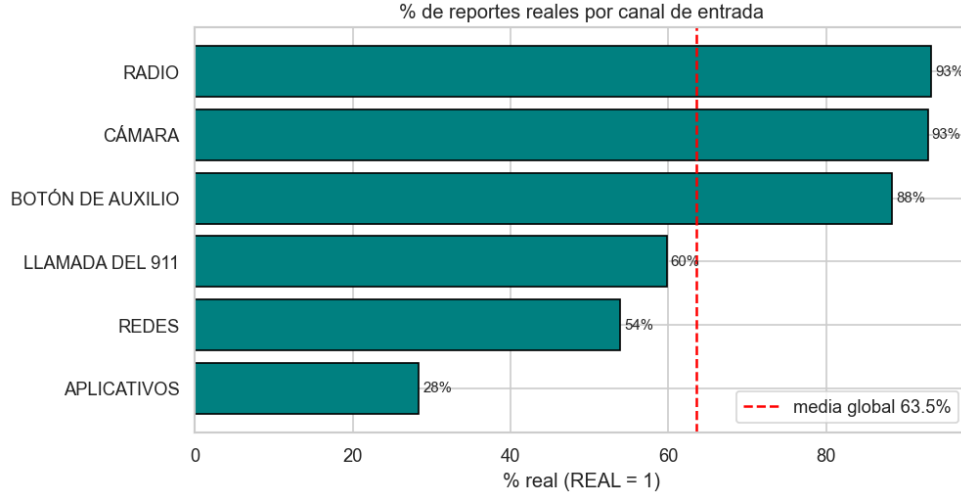


Figura 3: Tasa de confirmación (% de reportes reales) por canal de entrada.

Por alcaldía. El ranking va de Cuauhtémoc (75.3 %) y Azcapotzalco (72.4 %) hasta Xochimilco (51.8 %) y Cuajimalpa (50.7 %), sobre un rango de ~ 25 puntos (Figura 4). Las alcaldías centrales que cuentan con más cámaras, operadores y tráfico verificable tienden a mayor confirmación que la periferia. La brecha es real pero menor que la del canal (~ 33 puntos), por lo que la geografía sí aporta señal aunque sea complementaria.

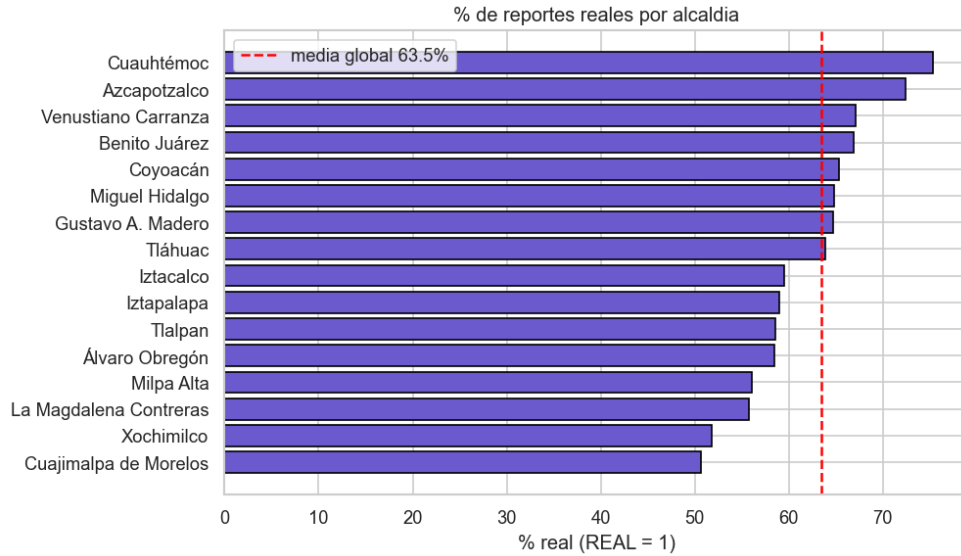


Figura 4: Tasa de confirmación por alcaldía.

Mezcla canal $\times$ alcaldía. La prueba χ^2 rechaza la independencia ($\chi^2 = 3719.8$, $gl = 120$, $p \approx 0$), pero con $n \approx 290k$ casi cualquier asociación resulta “significativa”. Lo que realmente es informativo es el tamaño de efecto, y la V de Cramér = 0.040 que es muy pequeña. La cámara pesa más en alcaldías centrales y el botón de auxilio en la periferia, pero el dominio de la llamada del 911 (entre el 80 % y el 95 % del volumen según la alcaldía) homogeneiza la mezcla. Es un buen ejemplo de que *estadísticamente significativo $\neq$ efecto grande*.

Por franja horaria. La confiabilidad decae a lo largo del día (Figura 5). En la madrugada

70.1 % y mañana 68.3 % superan la media global (63.5 %), mientras tarde (61.5 %) y noche (60.1 %) quedan por debajo, con una brecha de 10 puntos. Vemos entonces que en la tarde y noche se concentran además el mayor volumen (33.1 % y 33.6 %), pues las horas con más reportes son justamente las de menor proporción de incidentes reales. El patrón es coherente con el perfil horario de los canales donde en horas valle la vigilancia institucional (que confirma ~ 90 %) sostiene una proporción mayor del reporte, y la madrugada hereda parte de esa confiabilidad.

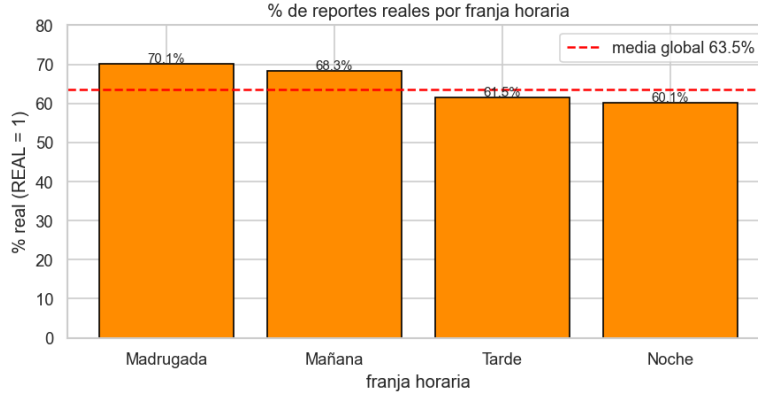


Figura 5: Tasa de confirmación por franja horaria.

4.5. Síntesis del EDA

- Target con desbalance moderado ($63.5/36.5$) $\Rightarrow$ F1 macro como métrica rectora y pesos de clase balanceados.
- El **canal de entrada** y el **tipo de incidente** separan fuertemente la veracidad (canales institucionales ~ 90 % vs. 911 ~ 60 %); se anticipan como señales clave del modelo.
- La **alcaldía** (rango 25 pts) y la **franja horaria** (rango 10 pts) aportan señal complementaria.
- Calidad de datos alta: faltantes marginales (≤ 0.13 %), sin duplicados tras la preparación; atípicos de tiempos de cierre documentados y justificados (no eliminados).
- Frontera anti-fuga explícita: lo que se conoce *al cerrar* el reporte no entra al modelo.

5. Modelo supervisado

5.1. Variable objetivo y variables de entrada

Se predice **REAL** (afirmativo vs. falso/informativo) usando *únicamente información disponible al recibir el reporte*: numéricas (HORA, MES, ANIO, latitud, longitud) y categóricas (tipo_incidente_c4, incidente_c4, tipo_entrada, alcaldia_catalogo, dia_semana, FRANJA). Se excluyen por fuga de información las columnas que solo se conocen tras el cierre (TIEMPO_CIERRE_MIN y clas_con_f_alarma, cuya categoría “FALSA ALARMA” es prácticamente el target) y los identificadores (folio, codigo_cierre, CIERRE_DESC, FECHA_CREACION). El efecto de la fuga se cuantifica en la Sección 5.6.

5.2. Proceso

Patrón Pipeline a nivel de modelo. El clasificador se arma como un Pipeline de `scikit-learn` [4] que encadena el preprocesamiento y el estimador en un solo objeto `ColumnTransformer` con imputación por moda + *one-hot* (`handle_unknown='ignore'`) para las categóricas y paso directo para las numéricas (un bosque corta por umbrales y no necesita escalado), seguido de un

RandomForestClassifier [5] con 300 árboles, `class_weight='balanced'` y semilla 42. Esto garantiza que el mismo preprocesamiento se aplique de forma idéntica en entrenamiento, validación y predicción.

Partición. 80/20 estratificada por el target con semilla fija: 231 908 reportes de entrenamiento y 57 977 de prueba (balance 63.5/36.5 en ambos lados).

Ajuste de hiperparámetros. GridSearchCV (validación cruzada estratificada de 3 pliegues, métrica F1 macro) sobre una submuestra estratificada de 80 000 reportes del entrenamiento, explorando profundidad y tamaño mínimo de hoja (Tabla 3). La mejor configuración (`max_depth= 20`, `min_samples_leaf= 1`) se reentrenó sobre el entrenamiento completo antes de la evaluación final.

Cuadro 3: Búsqueda de hiperparámetros (F1 macro, validación cruzada sobre submuestra de 80k).

<code>max_depth</code>	<code>min_samples_leaf</code>	F1 macro (CV)
20	1	0.6566
20	20	0.6548
None	20	0.6546
None	1	0.6207

5.3. Resultados

La Tabla 4 compara el modelo contra la línea base obligatoria (clasificador de clase mayoritaria) y contra una red neuronal entrenada como comparación (Sección 5.7).

Cuadro 4: Comparación de modelos en el conjunto de prueba (57 977 reportes).

Modelo	Accuracy	F1 macro	F1 pond.	AUC
Línea base (mayoritario)	0.635	0.388	0.493	0.500
Random Forest (final)	0.670	0.658	0.675	0.729
Red neuronal (MLP)	0.686	0.638	0.674	0.726

Comparación con la línea base (obligatoria). La línea base predice “real” para todo: alcanza 63.5 % de *accuracy* pero un F1 macro de 0.388 y no detecta *ningún* reporte falso, lo que la vuelve inservible para filtrar el ruido operativo. El Random Forest sube el F1 macro a 0.658 (+0.27) deteniendo una fracción sustancial de los falsos: el modelo aprende señal genuina y supera con holgura el piso trivial.

Métricas completas del Random Forest. Accuracy 0.6700; precisión / recall / F1 *macro*: 0.6578 / 0.6684 / 0.6581; *ponderadas*: 0.6898 / 0.6700 / 0.6753; AUC 0.7289. El desglose por clase (Tabla 5) muestra el equilibrio logrado con `class_weight='balanced'`: el recall de la clase minoritaria (falsos) llega a 0.66 sin colapsar el de los reales (0.67).

Cuadro 5: Desglose por clase del Random Forest (conjunto de prueba).

Clase	Precisión	Recall	F1	Soporte
Falso/Informativo (0)	0.539	0.663	0.594	21 164
Real/Afirmativo (1)	0.777	0.674	0.722	36 813

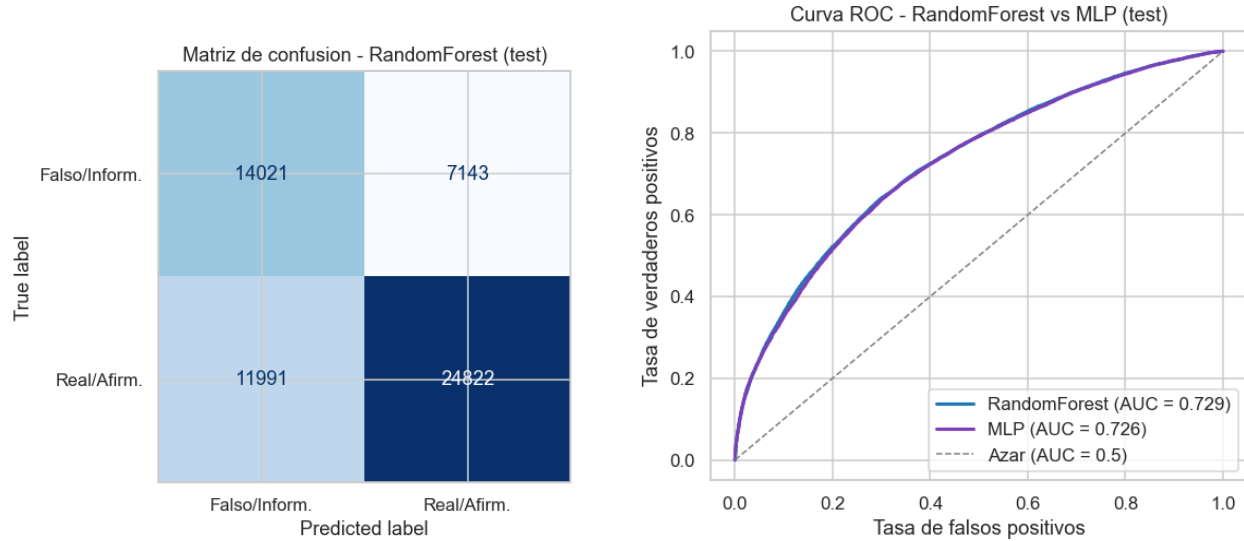


Figura 6: Izquierda: matriz de confusión del Random Forest. Derecha: curvas ROC del Random Forest (AUC = 0.729) y de la MLP (AUC = 0.726).

Interpretación. En la matriz de confusión la diagonal concentra la mayoría de los casos; los *falsos negativos* (reportes reales clasificados como falsos) son el error operativamente más costoso y se analizan más abajo. Ambas curvas ROC se despegan claramente del azar; el AUC moderado (~ 0.73) refleja el techo de separabilidad que impone la información disponible al momento del reporte: hay señal real y aprovechable, pero el problema no es trivialmente separable.

5.4. Importancia de variables

La importancia por permutación (muestra de 20 000 reportes de prueba, 5 repeticiones, métrica F1 macro; Figura 7) la dominan *incidente_c4* (0.102) y *tipo_entrada* (0.031), muy por encima del resto (≤ 0.006). Es decir: *qué* se reporta y *por qué canal* predice la veracidad mejor que *cuándo* o *dónde*. El resultado es coherente con el EDA: una cámara o un botón de auxilio suelen ir asociados a eventos confirmados, mientras que la llamada del 911 es mucho más heterogénea.

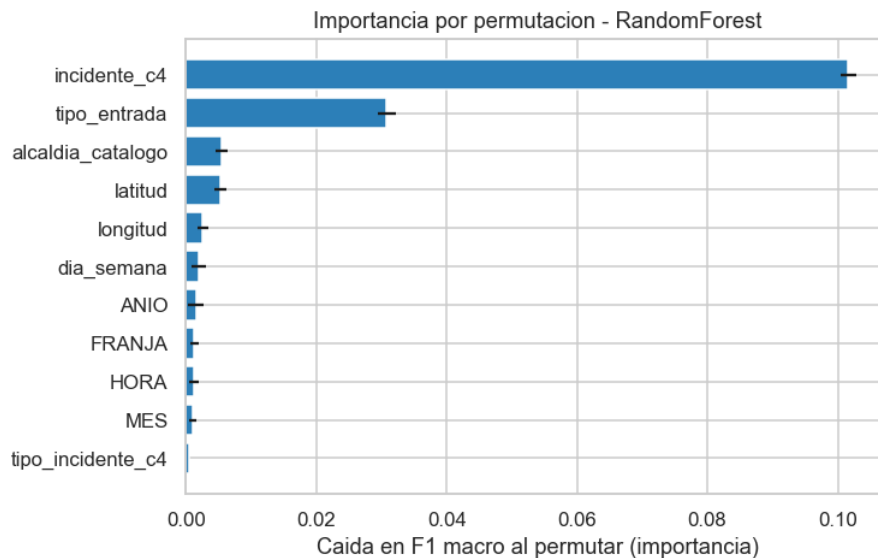


Figura 7: Importancia por permutación del Random Forest (caída del F1 macro al permutar cada variable original).

5.5. Análisis de errores

La tasa de error por canal (Tabla 6) confirma dónde vive la incertidumbre: la llamada del 911 se equivoca en el 36.3 % de los casos, mientras los canales verificados son mucho más predecibles (cámara 8.7 %, radio 7.1 %). De los 11 991 falsos negativos la inmensa mayoría proviene de llamadas al 911, que es a la vez el canal más voluminoso y el menos discriminante. La implicación es que el margen de mejora está en enriquecer la información de las llamadas ciudadanas (p. ej. con atributos del transcrito de la llamada), no en los canales ya confiables.

Cuadro 6: Tasa de error del modelo por canal de entrada (conjunto de prueba).

Canal	n	Tasa de error (%)
Llamada del 911	50 876	36.3
Botón de auxilio	2 853	11.8
Cámara	208	8.7
Radio	3 858	7.1
Aplicativos	103	2.9

5.6. Experimento de fuga de información

Para justificar empíricamente la exclusión anti-fuga, se reentrenó el *mismo* Random Forest sobre el *mismo* split añadiendo TIEMPO_CIERRE_MIN como variable: el F1 macro salta de 0.6581 a 0.7184 (+0.060). Vemos que hubo un salto, pero el tiempo de cierre solo se conoce después de resolver el caso, de modo que en uso real la columna no existiría y la mejora se evaporaría. El modelo honesto es el de la Tabla 4. Este experimento confirma que la frontera anti-fuga fue correcta.

5.7. Comparación con una red neuronal (MLP)

Como comparación se entrenó un `MLPClassifier` (capas 128–64, *early stopping*, semilla 42) sobre el mismo split, con el mismo preprocesamiento salvo el escalado de las numéricas (`StandardScaler`),

que el descenso de gradiente sí requiere. La MLP alcanza la mayor accuracy (0.686) y un AUC competitivo (0.726), pero su F1 macro (0.638) queda por debajo del Random Forest (0.658). Esto debido a que sin un mecanismo que compense el desbalance, la red se inclina hacia la clase mayoritaria (su recall de la clase “falso” cae a 0.44 frente al 0.66 del bosque), dejando escapar más reportes falsos clasificados como reales. Dado que el valor de negocio está justamente en filtrar los falsos para racionar los recursos de despacho, el Random Forest balanceado se mantiene como modelo final por desempeño en la métrica rectora e interpretabilidad. Ahora, en casos por donde atender la mayor cantidad de llamadas sea prioritario, la MLP podría ser una alternativa a considerar (este no es el caso).

5.8. Modelo final ligero y persistencia

El ganador del tuning (`min_samples_leaf=1`) producía un archivo serializado de ~ 700 MB, inviable de versionar en GitHub (límite de 100 MB). El modelo *que se persiste* se reentrenó con `min_samples_leaf=20` (mismos 300 árboles, profundidad 20, pesos balanceados) y un F1 macro en prueba de 0.6561, con una diferencia (-0.002) básicamente inexistente a cambio de un archivo de **43.1 MB** (`models/modelo_rf.joblib`). La regularización por tamaño de hoja poda el bosque sin degradar el desempeño siendo casi igual de bueno, pero mucho más ligero y versionable.

6. Agrupamiento (Clustering)

6.1. Propósito y elección del algoritmo

Cambiamos de pregunta: en lugar de *predecir* si un reporte se confirma, buscamos estructura geográfica. Nos preguntamos, **¿en qué vialidades de la CDMX se concentran los incidentes y de qué tipo son?**. Se usa **DBSCAN** [6], que agrupa por *densidad*, por tres principales razones:

- encuentra zonas densas de forma libre (un corredor vial es una nube *alargada* que sigue el trazo de una avenida. Con K-Means solo encontraríamos grupos aproximadamente esféricos y nunca capturaría un corredor lineal).
- no exige fijar k . El número de corredores *emerge* de la densidad de los datos (desconocemos cuantos hay).
- separa el ruido en lugar de forzarlos a un grupo, y ese ruido es señal, no error. El target **REAL** *no* participa en el agrupamiento y se usa solo a posteriori para caracterizar los grupos (validación externa).

6.2. Preprocesamiento y selección de parámetros

Proyección a metros. Sobre latitud/longitud en grados, ε no tiene un significado físico claro (a la latitud de la CDMX un grado de longitud y uno de latitud miden distinto). Se proyecta con una equirectangular local centrada en la ciudad ($\text{lat}_0 = 19.36$, $\text{lon}_0 = -99.13$), de modo que ε (la distancia de **DBSCAN**) queda **en metros sobre el asfalto** y es directamente interpretable.

Curva de k -distancias. La distancia de cada punto a su k -ésimo vecino ($k = \text{min_samples}$), calculada sobre las 289 885 filas completas (una submuestra bajaría la densidad local y desplazaría el codo), es plana y baja solo en su primer tramo donde el $\sim 12\%$ de los puntos más densos tiene a su vecino 200 a ≤ 150 m y enseguida se dispara. Ese despegue es el codo. Se eligen $\varepsilon = 150$ m (el ancho típico de un crucero o vialidad densa) y `min_samples = 200`, deliberadamente alto ya que no se buscan micro-cúmulos de cuatro choques sino *corredores con masa crítica* (≥ 200 incidentes en

2022–2024). Ahora, debido a que ε es interpretable, podríamos jugar buscando ahora no vialidades, sino cruces peligrosos reduciendo ε y la cantidad mínima de choques en la zona.

6.3. Resultados

DBSCAN encuentra **168 cúmulos densos**, deja 223 958 puntos (77.3 %) como ruido y 65 927 incidentes (22.7 %) caen en alguna estructura densa. El ruido alto no es un fallo del método, puede significar que tras eliminar los duplicados, la mayoría de los incidentes ocurre *fuera* de corredores densos, repartida por toda la traza urbana. Esto es algo que esperamos pues estos incidentes pueden suceder en cualquier lugar, pero si notamos que existe una minoría “concentrada” atacable por corredores concretos y una mayoría “difusa” que exige cobertura general.

Morfología: corredores vs. cruceros. Para cada cluster del top-12 se midió con PCA la *elongación* ($\sqrt{\text{var}_1/\text{var}_2}$) y la *longitud* sobre el eje principal, viendo que solo 1 alcanza la forma de corredor lineal ($\text{elongación} \geq 3$ y ≥ 1 km) y 11 son cruceros o zonas compactas. La deduplicación adelgaza las líneas y deja sobre todo manchas densas alrededor de nodos. Con esto podemos tomar medidas particulares, pues mientras que a un crucero lo cubrimos con una grúa o patrulla fija, un corredor exige patrullaje a lo largo del tramo.

Nombres de vialidad. Los centroides del top se cruzaron con un catálogo geocodificado previamente, asignando el registro más cercano en metros con un umbral de 1 000 m. En caso de que el registro quede más lejos, se usa una etiqueta genérica por alcaldía antes que arriesgar un nombre equivocado. La Tabla 7 resume los 12 nodos principales.

Cuadro 7: Los 12 nodos densos más grandes (DBSCAN, $\varepsilon = 150$ m, `min_samples=200`). El incidente dominante en todos es *choque sin lesionados*. La franja pico cae en tarde/noche.

Vialidad / zona	n	% real	Alcaldía	Morfología
Calle 73	1 645	59.6	Venustiano Carranza	crucero/zona
Calle Mendelssohn	1 534	60.2	Gustavo A. Madero	crucero/zona
Iztapalapa (19.317, −99.075)	1 143	51.0	Iztapalapa	corredor
Calle Tiziano	935	59.3	Benito Juárez	crucero/zona
Circuito Interior Melchor Ocampo	912	58.1	Miguel Hidalgo	crucero/zona
Avenida 4	900	59.9	Venustiano Carranza	crucero/zona
Zona densa (Álvaro Obregón)	854	53.9	Álvaro Obregón	crucero/zona
Distribuidor Vial San Antonio	785	60.8	Benito Juárez	crucero/zona
Zona densa (Álvaro Obregón)	777	45.9	Álvaro Obregón	crucero/zona
Calz. General Ignacio Zaragoza	756	47.8	Iztacalco	crucero/zona
Zona densa (Iztapalapa)	725	47.9	Iztapalapa	crucero/zona
Zona densa (Coyoacán)	723	53.4	Coyoacán	crucero/zona

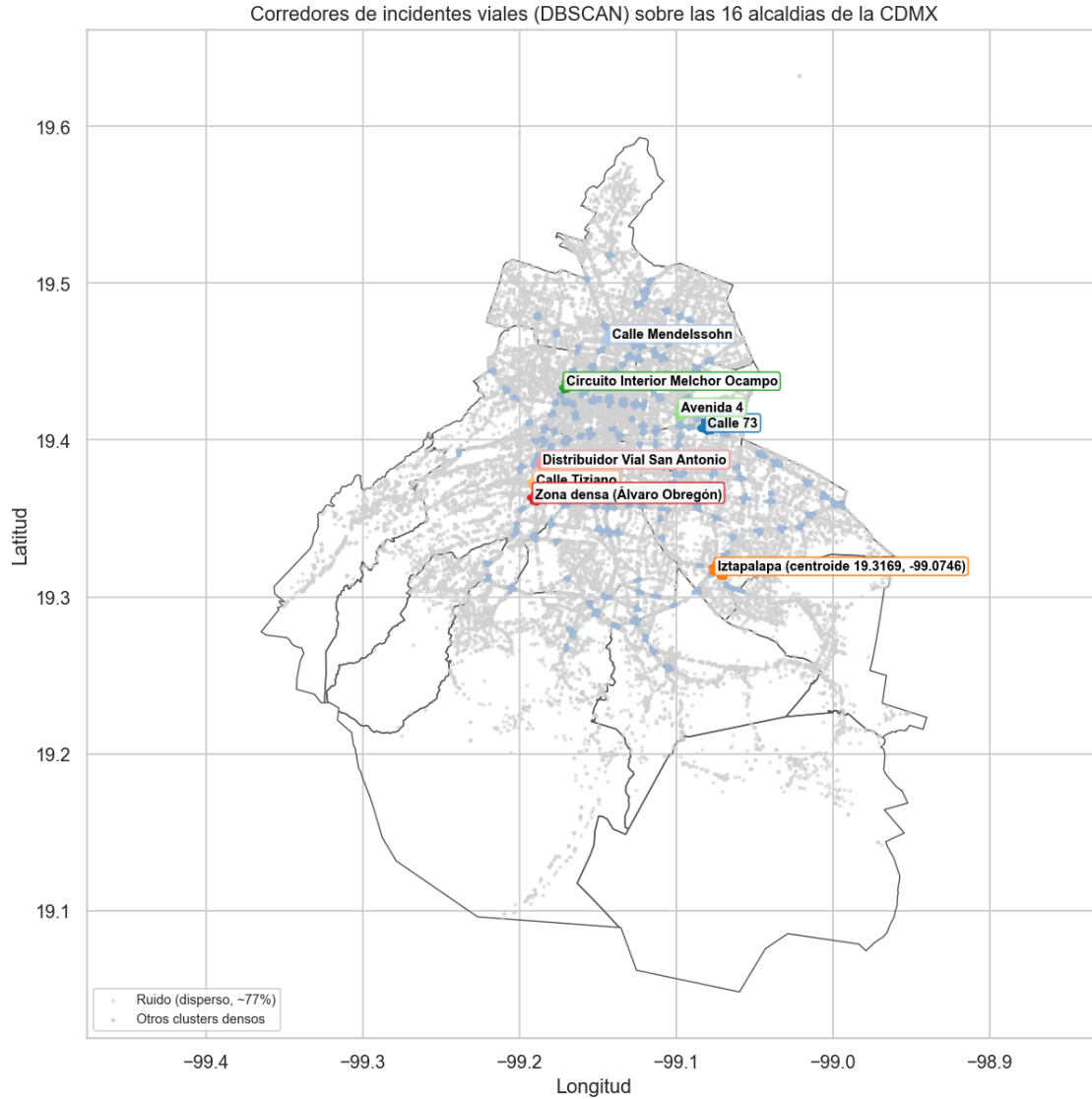


Figura 8: Nodos densos de incidentes sobre la silueta de la CDMX (contorno oficial de alcaldías). El gris es ruido (incidentes dispersos) y los colores son clusters de DBSCAN, con los principales etiquetados por su vialidad. La versión **interactiva** (zoom, paneo y *hover* con vialidad, tipo, alcaldía y % real) está en figures/mapa_corredores.html.

6.4. Evaluación e interpretación

Cohesión interna. El coeficiente de silueta sobre los puntos no-ruido (muestra de 20 000, semilla 42) es **0.753**. Los puntos densos están, en promedio, mucho más cerca de su propio cluster que del vecino. Los grupos tienen cohesión y separación reales y no son cortes arbitrarios.

Validación externa con el target. Como REAL nunca participó en el agrupamiento, comparar el % afirmativo por cluster contra el global (63.5 %) es una validación externa genuina, con dos hallazgos. Primero, entre los 165 clusters con $n \geq 200$ el % afirmativo se abre en una **banda amplia** de 40.8 % a 89.7 % (mediana 59.2 %; 52 por encima del global y 113 por debajo), es decir, la *vialidad concreta* sí lleva señal sobre la veracidad del reporte. Segundo, los nodos densos confirman *por debajo* del global (58.9 % dentro de clusters vs. 64.9 % en el ruido): las vialidades de altísimo

flujo acumulan proporcionalmente más reportes falsos/informativos (saturación, llamadas repetidas, sobre-reporte en hora pico). De aquí podemos concluir que un reporte sobre una vialidad saturada merece *más* verificación, no menos (esperabamos lo contrario).

7. Arquitectura y diseño

Organización del código (POO). El código fuente vive en el paquete `src/`, separado de los notebooks, que solo lo importan y lo usan. Está organizado por responsabilidad: `src/preparacion.py` (limpieza de datos), `src/modelado.py` (clasificadores), `src/clustering.py` (agrupamiento), `src/viz.py` (graficado) y `src/config.py` (semilla, rutas y listas de variables). El diseño usa **herencia** donde es natural: los pasos de limpieza son subclases de `PasoLimpieza`, y `ModeloRandomForest` y `ModeloMLP` heredan de `ModeloClasificador` (comparten el flujo `prep` → `clf` y solo cambian el tratamiento numérico y el estimador), y **composición** donde corresponde: `PipelinePreparacion` contiene una lista de pasos. La Figura 9 muestra el diagrama UML de clases.

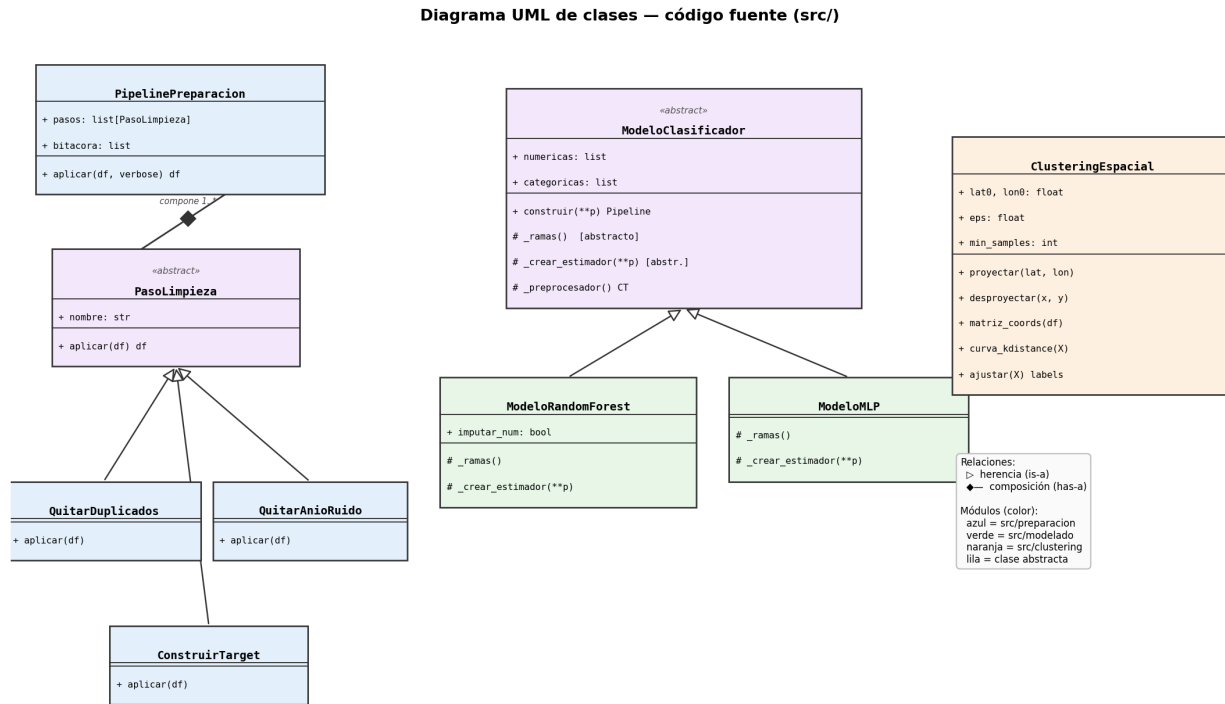


Figura 9: Diagrama UML de clases del código fuente (`src/`): herencia (triángulo hueco) en los pasos de limpieza y en los clasificadores, y composición (rombo) entre `PipelinePreparacion` y `PasoLimpieza`.

Patrón de diseño implementado: Pipeline, en dos niveles.

1. Preparación de datos

La clase `PipelinePreparacion` encadena pasos atómicos de limpieza (`QuitarDuplicados` → `QuitarAnioRuido` → `ConstruirTarget`, subclases de `PasoLimpieza`) y registra una bitácora auditable (Tabla 2); agregar, quitar o reordenar un paso es trivial y la trazabilidad queda impresa.

2. Modelado

El Pipeline de `scikit-learn` (construido por las clases de `src/modelado.py`) encadena codificación → clasificador en un único objeto, garantizando que el preprocesamiento ajustado en entrenamiento se aplique idéntico en validación, prueba y producción que a su vez previene fugas de preprocesamiento. Se eligió Pipeline porque el procesamiento del proyecto ocurre naturalmente en etapas secuenciales bien definidas (limpieza → codificación → modelo), y el patrón maximiza la reproducibilidad y la legibilidad del flujo.

Diagrama de flujo. La Figura 10 muestra el flujo completo: el pipeline de preparación (1.) produce el dataset de trabajo que consumen los tres notebooks, y el pipeline de modelado (2.) encapsula preprocesamiento y clasificador en el objeto único que se persiste y recarga.

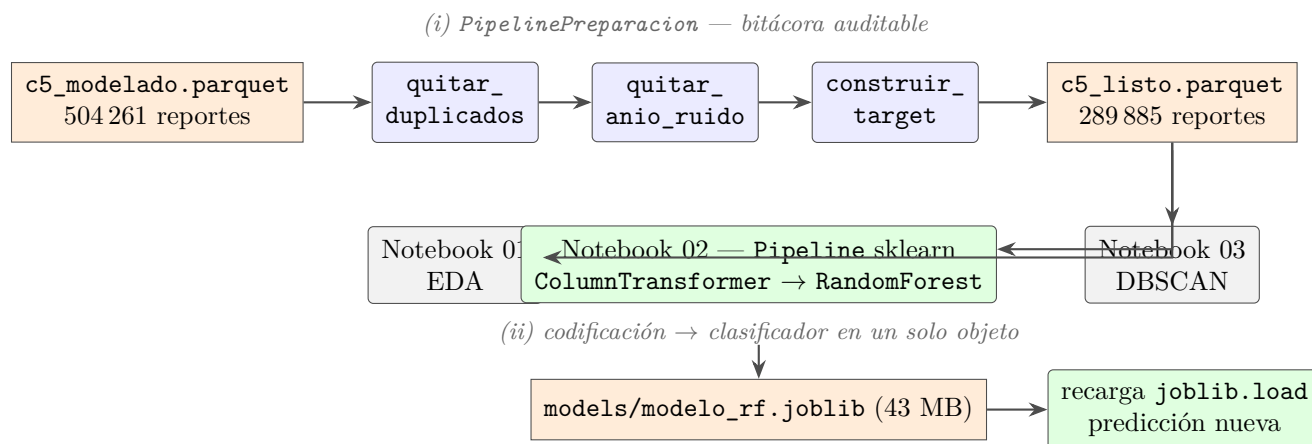


Figura 10: Arquitectura del proyecto: el patrón Pipeline en sus dos niveles y la persistencia/recarga del modelo.

8. Reutilización del modelo

El modelo final se serializa con `joblib` en `models/modelo_rf.joblib` (43.1 MB). La última sección del notebook `02_clasificacion.ipynb` lo **recarga desde disco** simulando un servicio que consume el archivo sin reentrenar y predice reportes nuevos contruidos a mano. La Tabla 8 muestra la salida real de la demostración. El modelo discrimina de forma coherente con el EDA (canales institucionales y zonas centrales al alza, llamada del 911 en la periferia y de madrugada a la baja).

Cuadro 8: Demostración de recarga: predicciones sobre tres reportes nuevos.

Canal	Alcaldía	Incidente	Franja	P(real)	Clase
Cámara	Cuauhtémoc	Choque con lesionados	Tarde	0.806	REAL
Llamada del 911	Tláhuac	Choque sin lesionados	Madrugada	0.355	falso/inform.
Botón de auxilio	Benito Juárez	Choque con lesionados	Mañana	0.895	REAL

9. Conclusiones

- **El qué y el cómo del reporte predicen su veracidad.** El tipo de incidente y el canal de entrada concentran la señal del modelo (importancias por permutación 0.102 y 0.031); en el

análisis bivariado, los canales institucionales confirman $\sim 90\%$ frente al $\sim 60\%$ de la llamada del 911, que es el 87.6% del volumen.

- **El modelo supera con holgura a la línea base** (F1 macro $0.39 \rightarrow 0.66$; AUC 0.73), deteniendo dos de cada tres reportes falsos sin colapsar el recall de los reales. El techo de desempeño refleja la información limitada disponible al momento del reporte, no una falla del algoritmo: la red neuronal comparada empata en AUC y pierde en F1 macro.
- **La frontera anti-fuga importa y es demostrable**: incluir el tiempo de cierre infla el F1 macro en $+0.06$ de forma espuria; el experimento lo cuantifica y justifica el diseño.
- **La confiabilidad tiene geografía y horario**: decae del centro a la periferia (~ 25 pts entre alcaldías) y de la madrugada a la noche (10 pts), siendo las horas de mayor volumen las menos confiables.
- **Los incidentes tienen estructura geográfica accionable, pero acotada**: DBSCAN encuentra 168 nodos densos (silueta 0.753) que concentran solo el 22.7% de los incidentes —cruceos y ejes con nombre propio (Circuito Interior, Distribuidor Vial San Antonio, Calz. Ignacio Zaragoza)— mientras el 77% restante está disperso. Y la validación externa revela un matiz contraintuitivo: las vialidades más saturadas confirman *por debajo* del promedio (58.9% vs. 64.9% del ruido), de modo que un reporte en una vialidad densa merece más verificación, no menos.
- **Limitaciones**: etiquetado institucional del cierre; ventana 2022–febrero 2024; sin variables de contexto (clima, tráfico, contenido de la llamada). **Trabajo futuro**: enriquecer las llamadas del 911 (donde se concentra el error), calibrar el umbral de decisión según el costo operativo del despacho y explorar una zona de abstención para derivar los casos ambiguos a verificación humana.

10. Uso de Inteligencia Artificial Generativa

A lo largo de todas las tareas experimentamos con varios modelos de GenAI. De todos esos experimentos, encontramos que Claude es la que nos dio mejores resultados. Para este proyecto, y debido a que solo soy un integrante, y el proyecto es bastante similar a las tareas y el examen 4 (que por suerte fue de **DBSCAN**), la mayoría del código escrito y redacción de documentos fue realizada por Claude, más específicamente Claude Code a través de ultracode para la subdelegación de tareas con modelos apropiados y pasando como contexto todo el análisis, medidas, procedimientos, etc. de las tareas anteriores. Para las tareas que requerían buscar o arreglar alguna gráfica o bug, se le indico al coordinador asignar estas tareas al modelo Opus 4.8, y una vez identificado el problema se subdelegaba la implementación a Sonnet para ahorro de tokens manteniendo siempre una verificación. También se hicieron ciertos experimentos con Fable 5, pero no notamos gran diferencia respecto al desempeño de Opus 4.8, pero el consumo de tokens era absurdo, por lo que casi no se utilizo.

Vale la pena aclarar el reparto de responsabilidades. La inteligencia artificial funcionó como una herramienta de ejecución, es decir, la escritura de código, la primera redacción de los textos y la depuración, siempre bajo mi dirección. En cambio, las decisiones analíticas más importantes, como la definición de la variable objetivo, la eliminación de los duplicados, la elección de la métrica F1 macro y la interpretación de los resultados, fueron supervisadas y validadas por mí. Revisé de forma crítica cada propuesta del modelo antes de adoptarla, y varias las corregí o descarté cuando no me convencían.

Respecto al uso de tokens, todo se hizo sobre una única sesión de en total 25 horas, llegando a un consumo de casi 3.74 millones de tokens. Es una cantidad absurda, que se tuvo que repartir a lo

largo de dos semanas en sesiones de 5 horas. La distribución de uso fue:

Usage by model:

```
-opus-4-8: 153.5k input, 2.3m output, 268.1m cache read, 9.8m cache write
-haiku-4-5: 957.0k input, 27.6k output, 0 cache read, 0 cache write, 49 web search
-fable-5: 36.4k input, 345.0k output, 30.2m cache read, 2.4m cache write
-sonnet-4-6: 9 input, 1.8k output, 78.1k cache read, 15.1k cache write
```

Por otro lado, todo el texto del reporte, cuarto, notebooks y presentación fue revisado y editado por mí, llevando también la coordinación de que reportes se podían aplicar sobre el dataset y que medidas y desiciones tomar sobre los modelos. Un ejemplo de esto fue la duplicación de reportes. En el momento donde se le pidió al agente que entendiera el dataset, identificó que existían reportes duplicados, pero la decisión que tomo respecto a estos fue marcarlos como falsos. Esto provocaba que el entrenamiento de los modelos fuera incorrecto ya que de cierta forma se tenían reportes verdaderos y falsos de un mismo incidente.

Referencias

- [1] Gobierno de la Ciudad de México, C5. (2024). *Incidentes viales reportados por el C5* [conjunto de datos]. Portal de Datos Abiertos de la CDMX. <https://datos.cdmx.gob.mx>
- [2] Melgoza, E., Beltrán-Sánchez, H., & Vargas Bustamante, A. (2023). Injury-related emergency medical service calls, traffic accidents, and crime in Mexico City before and during the COVID-19 pandemic. *Prehospital and Disaster Medicine*, 38(1), 73–80. <https://doi.org/10.1017/S1049023X22002230>
- [3] Herrera Ortiz, M. L., Pérez Ferrer, C., Quintero Valverde, C., Chías Becerril, L., Martínez Santiago, A., Reséndiz López, H. D., Quistberg, D. A., & Barrientos Gutiérrez, T. (2025). Trends in collisions and traffic mortality rates in Mexico City: A comparison of six data sources. *PLOS ONE*, 20(10), e0334103. <https://doi.org/10.1371/journal.pone.0334103>
- [4] Pedregosa, F., Varoquaux, G., Gramfort, A., et al. (2011). Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830. <https://jmlr.org/papers/v12/pedregosa11a.html>
- [5] Breiman, L. (2001). Random Forests. *Machine Learning*, 45(1), 5–32. <https://doi.org/10.1023/A:1010933404324>
- [6] Ester, M., Kriegel, H.-P., Sander, J., & Xu, X. (1996). A Density-Based Algorithm for Discovering Clusters in Large Spatial Databases with Noise. *Proceedings of KDD-96*, 226–231.
- [7] McKinney, W. (2010). Data Structures for Statistical Computing in Python. *Proceedings of the 9th Python in Science Conference*, 56–61. <https://doi.org/10.25080/Majora-92bf1922-00a>